

THE FIRST BOOSTING ALGORITHM

AdaBoost

Many very small trees, built one after another. Each one works on the rows the last one got wrong, and each one carries a weight in the final answer.

The Flow We Will Follow

- 01 The two ensemble families, and where AdaBoost sits
- 02 A full decision tree, and why boosting starts from the other end
- 03 The decision stump, and the weighted formula AdaBoost builds
- 04 Six steps of maths, worked out on a seven-row dataset
- 05 One full prediction on a new row
- 06 What changes for a regression problem

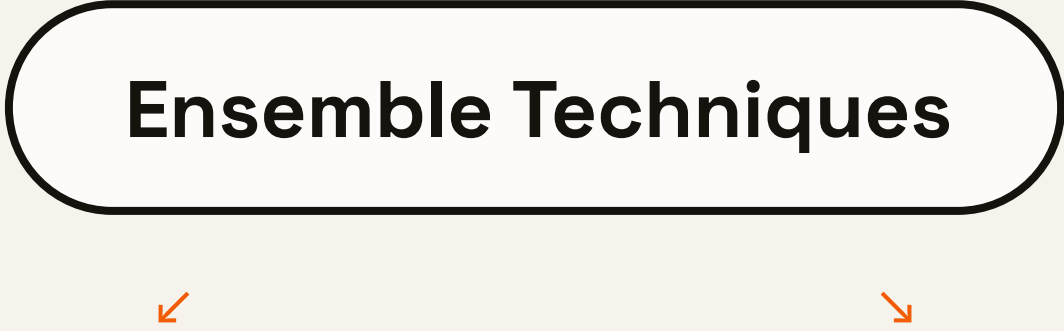
THE BOOSTING FAMILY

- 1 AdaBoost today
- 2 Gradient Boosting
- 3 XGBoost

All three use decision trees inside, and all three solve classification and regression.

Two Families of Ensemble Techniques

Ensemble Techniques



```
graph TD; A[Ensemble Techniques] --> B[Bagging]; A --> C[Boosting]
```

Bagging { base learners }

Models train side by side, each on its own sample of the data.

-
- 1 Random Forest Classifier
 - 2 Random Forest Regressor

Boosting { weak learners }

Models are connected one after another, in a line.

-
- 1 AdaBoost today
 - 2 Gradient Boosting
 - 3 XGBoost

In both families the model inside is a decision tree, and both handle classification and regression.

A Full Tree Overfits. A Forest Fixes the Variance.

DECISION TREE, FULL DEPTH

Train Acc ↑ Test Acc ↓

That is **overfitting**.

low bias high variance



many trees,
many samples,
votes counted

RANDOM FOREST · BAGGING

low bias low variance

The bias stays, the variance comes down.
That is a **generalised model**.

Classification takes the majority vote,
regression takes the average of the outputs.

Hold on to this picture. The stump we are about to build sits at the exact opposite end.

Weak Learners in a Line Make a Strong Learner



Each model gets the rows the previous one got wrong. DT 2 cannot start before DT 1 is done.

Weak learner means it has not learnt much from the training dataset.

■ THE KEY IDEA

AdaBoost Assigns Weights to the Weak Learners

$$F = \alpha_1(M_1) + \alpha_2(M_2) + \alpha_3(M_3) + \dots + \alpha_n(M_n)$$

$M_1 \dots M_N$

The decision tree stumps, our weak learners.

$\alpha_1 \dots \alpha_N$

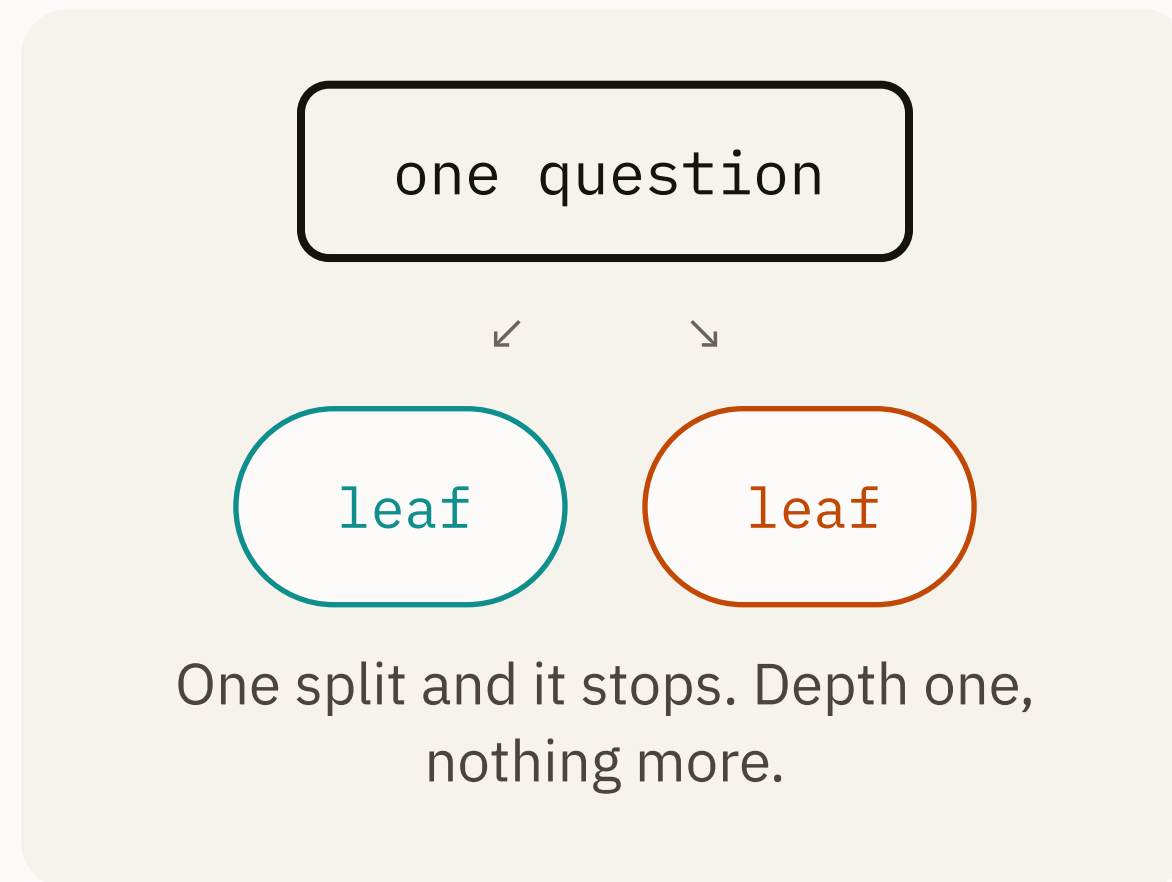
The weights. One number per stump, telling us how much it counts.

SAME FORMULA FOR

Classification and regression, both.

No majority voting here. Everything we calculate from now on is either an M or an α in this one line.

A Decision Tree Stump Has Depth = 1



ON ITS OWN IT UNDERFITS

Train Acc ↓ 40% Test Acc ↑ 45%

Test is higher, but training itself is poor. The model has not learnt the data.

one stump high bias low
variance



many stumps joined low bias
high variance

One stump is too simple to be useful. Joined up with weights, the combination becomes a proper model.

Seven Applicants for an Internship

#	EXPERIENCE	PORTFOLIO	SHORTLISTED
1	≤ 1 yr	Weak	No
2	≤ 1 yr	Good	Yes
3	> 1 yr	Weak	No
4	≤ 1 yr	Good	Yes
5	≤ 1 yr	Normal	No
6	> 1 yr	Normal	Yes
7	> 1 yr	Good	Yes

Two features. **Experience** is up to one year or more than one year. **Portfolio** is weak, good or normal.

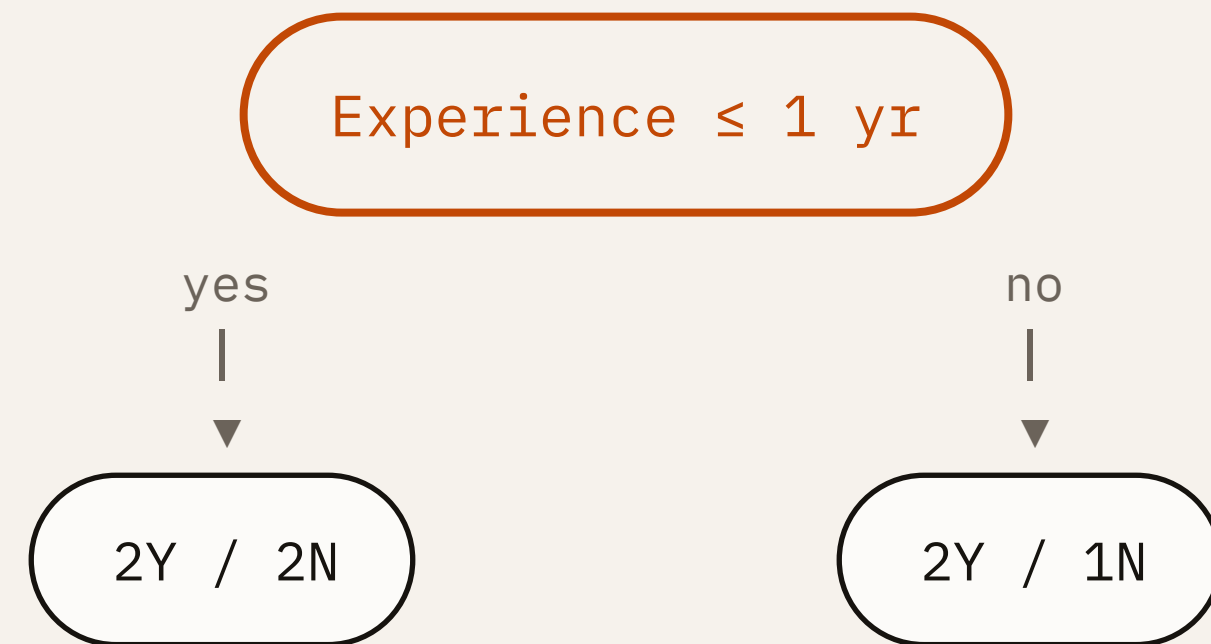
The target is **Shortlisted** . Four yes and three no in seven rows.

A good portfolio always means yes. A portfolio that is not good mostly means no, and **row 6 is the exception** . Keep an eye on it.

STEP 1

Create the Stumps, One per Feature

STUMP A · EXPERIENCE



Both sides are mixed. Neither leaf is clean.

STUMP B · PORTFOLIO



The left leaf is completely clean, the right one nearly.

We can build many more stumps than these two. To pick the best one we use **entropy** or **Gini impurity**.

Select the Best Stump With Entropy

STUMP A • EXPERIENCE

2Y / 2N $\rightarrow$ E = 1.000 2Y / 1N $\rightarrow$ E = 0.918

A fifty-fifty leaf is the most impure there is: entropy 1, Gini 0.5.

$$(4/7)(1.000) + (3/7)(0.918) = 0.965$$

STUMP B • PORTFOLIO

3Y / 0N $\rightarrow$ E = 0.000 1Y / 3N $\rightarrow$ E = 0.811

A pure leaf has entropy zero. Nothing is left to split.

$$(3/7)(0.000) + (4/7)(0.811) = 0.463$$

$$H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

Lower entropy is better.
0.463 beats 0.965, so
stump B is selected.

Best entropy means it
reaches a pure leaf fastest.
This stump is our **M₁**.

Gini impurity gives the same
winner. Use either one.

Sample Weights and the Total Error

#	EXP	PORTFOLIO	SHORT .	SAMPLE W	STUMP
1	≤ 1 yr	Weak	No	1/7	✓
2	≤ 1 yr	Good	Yes	1/7	✓
3	> 1 yr	Weak	No	1/7	✓
4	≤ 1 yr	Good	Yes	1/7	✓
5	≤ 1 yr	Normal	No	1/7	✓
6	> 1 yr	Normal	Yes	1/7	✗ error
7	> 1 yr	Good	Yes	1/7	✓

Seven records, so every sample weight is 1/7. They add up to 1.

WHAT THE STUMP SAYS

Good → Yes not Good → No

Row 6 is not good, so it says No. The real answer is Yes.

SUM OF THE TOTAL ERRORS

Add the sample weights of the wrong rows.

TE = 1/7 ≈ 0.1428

■ STEP 2, PART TWO

Performance of the Stump

THE FORMULA

$$\frac{1}{2} \cdot \ln \left[\frac{(1 - \text{TE})}{\text{TE}} \right]$$

$$= \frac{1}{2} \cdot \ln \left[\frac{(1 - 1/7)}{(1/7)} \right] = \frac{1}{2} \cdot \ln \left[\frac{(6/7)}{1/7} \times 7 \right] = \frac{1}{2} \cdot \ln [6]$$

$$\alpha_1 \approx 0.896$$

Small error $\rightarrow$ big fraction $\rightarrow$ big log $\rightarrow$ **big α** . Few mistakes earns a big say.

If TE were 0.5, the fraction is 1 and the log is 0, so **$\alpha = 0$** . A coin-toss stump gets no say.

If TE goes above 0.5, α turns negative, which means we take the opposite of what that stump says.

$$F = 0.896(M_1) + \alpha_2(M_2) + \dots \text{ first weight filled in}$$

Update the Weights

#	EXP	PORTFOLIO	SHORT.	SAMPLE W		UPDATED W
1	≤ 1 yr	Weak	No	1/7 ↓	→	0.058
2	≤ 1 yr	Good	Yes	1/7 ↓	→	0.058
3	> 1 yr	Weak	No	1/7 ↓	→	0.058
4	≤ 1 yr	Good	Yes	1/7 ↓	→	0.058
5	≤ 1 yr	Normal	No	1/7 ↓	→	0.058
6	> 1 yr	Normal	Yes	1/7 ↑	→	0.349
7	> 1 yr	Good	Yes	1/7 ↓	→	0.058

FOR CORRECT CLASSIFIED POINTS

= weight × e^{-(perf of stump)} = (1/7) × e^{-0.896} = 0.058

FOR INCORRECT CLASSIFIED POINTS

= weight × e^{+(perf of stump)} = (1/7) × e^{+0.896} = 0.349

Only the **sign** changes between the two.
Row 6 is now about six times heavier than the rest.

Normalised Weights, Then Bins

#	PORTFOLIO	UPDATED W	NORMALISED	BIN
1	Weak	0.058	0.08	0.00 – 0.08
2	Good	0.058	0.08	0.08 – 0.16
3	Weak	0.058	0.08	0.16 – 0.24
4	Good	0.058	0.08	0.24 – 0.32
5	Normal	0.058	0.08	0.32 – 0.40
6	Normal	0.349	0.50	0.40 – 0.90
7	Good	0.058	0.08	0.90 – 0.98
	total	0.697	≈ 1	

The updated weights add to 0.697 , not 1.
So divide every weight by 0.697.

$0.058 / 0.697 = 0.08$ $0.349 / 0.697 = 0.50$



Each row gets a slice of the 0 to 1 ruler as wide as its weight. Row 6 owns **half the ruler** .

Select the Rows for the Next Stump

RANDOM	ROW	EXP	PORTFOLIO	SHORT.
0.50	6	> 1 yr	Normal	Yes
0.10	2	≤ 1 yr	Good	Yes
0.60	6	> 1 yr	Normal	Yes
0.75	6	> 1 yr	Normal	Yes
0.22	3	> 1 yr	Weak	No
0.30	4	≤ 1 yr	Good	Yes
0.87	6	> 1 yr	Normal	Yes

An iterative process: pick a random value between 0 and 1, find its bin, copy that row. Repeat until the new table has seven rows.

0.50, 0.60, 0.75, 0.87 → row 6
0.10 → row 2 0.22 → row 3 0.30 → row 4

Row 6 got picked **four times out of seven**, and three rows were never picked.

Nobody told it to prefer row 6. It happens because row 6 owns half the ruler.

■ STEP 6

Send the New Table to Stump 2, Then Repeat

new 7-row
table



sample weights
1/7



best stump by
entropy



total error



$\alpha_2 = 0.65$



update,
normalise, bins

$$F_1 = 0.896(M_1) + 0.65(M_2)$$

STEPS 2 TO 6 REPEAT

Each round fills in one more α and one more M .

HOW MANY STUMPS

As many as we ask for. Default in scikit-learn: `n_estimators = 50`

.

WHY IT IMPROVES

Stump 2 trained on a table full of row 6, so stump 2 gets row 6 right.

STEP 7

Final Prediction on a New Row

Test data (Experience $\leq$ 1 yr , Portfolio = Good)

DTS 1

Yes

$$\alpha_1 = 0.896$$

DTS 2

No

$$\alpha_2 = 0.650$$

DTS 3

Yes

$$\alpha_3 = 0.240$$

DTS 4

No

$$\alpha_4 = -0.30$$

$$F = \alpha_1(M_1) + \alpha_2(M_2) + \alpha_3(M_3) + \alpha_4(M_4) = 0.896(\text{Yes}) + 0.650(\text{No}) + 0.240(\text{Yes}) - 0.30(\text{No})$$

PERFORMANCE OF SAY · YES

$$0.896 + 0.240 = 1.136$$

PERFORMANCE OF SAY · NO

$$0.650 - 0.30 = 0.350$$

1.136 is bigger than 0.350, so the output is **Yes**. This applicant gets shortlisted.

We added weights, not votes. The last stump's α was negative, so its No pulled the No side down.

Same Applicants, Now Predicting Stipend

Target = monthly stipend in ₹ thousands (a number, not Yes / No)

DTS 1

18

$\alpha_1 = 0.9$

DTS 2

22

$\alpha_2 = 0.6$

DTS 3

20

$\alpha_3 = 0.3$

EACH LEAF HOLDS A NUMBER

The leaf value is the **average stipend** of the rows that reached it, so the stump outputs a number instead of Yes or No.

WEIGHTED AVERAGE, NOT A PLAIN ONE

$$(0.9)(18) + (0.6)(22) + (0.3)(20) \div (0.9 + 0.6 + 0.3)$$

$$= (16.2 + 13.2 + 6.0) / 1.8 = 35.4 / 1.8$$

Predicted stipend ≈ 19.7

A plain average would give 20. Stump 1 had the biggest α and it said 18, so the answer got pulled towards 18.

Only Two Things Change

1 • HOW THE STUMP IS SELECTED

Entropy and Gini cannot be used, because the target is a number.

entropy → **mean squared error**

The same measure we used in the decision tree regressor.

2 • WHAT THE LEAVES HOLD

Numbers instead of Yes and No.

So the final answer is a weighted combination of those numbers, not a comparison of two sides.

EVERYTHING ELSE IS THE SAME

Sample weights, total error, performance of the stump, update, normalise, bins, pick the next rows.

AdaBoostClassifier

AdaBoostRegressor

QUESTION

How Do We Know a Record Is Wrong?

SHORT ANSWER

The training data **already has the true answer** in the last column. We just compare.

```
row 6, real answer Yes stump predicted No  
different → wrong row
```

The machine does this for all seven rows by itself. This is supervised learning, so the answers were given to us.

No domain expert is needed at this point. The expert mattered **earlier**, when someone had to record which past applicants were actually shortlisted.

Once that column exists, the algorithm only checks prediction against label. Nothing is judged by hand during training.

Be careful though: if a **label itself is wrong**, AdaBoost keeps raising its weight and chases a mistake that was never real. That is why AdaBoost is known to be sensitive to noisy data.

QUESTION

Is the Random Number Really Random, and Only 7 Times?

YES, AND YES

It is a plain random number between 0 and 1, the ordinary kind. There is no cleverness in the number.

The cleverness is in the **bins**, which are wide for the hard rows and thin for the easy ones.

```
7 rows → 7 draws 1000 rows → 1000 draws
```

One draw per row we need, so the new table stays the same size as the old one.

The picking allows **repetition**, so the same row can come up many times and some rows will not come up at all. That is exactly what we saw with row 6.

Because it is random, two runs of the same code can give slightly different tables.

Want the same result every time? Set `random_state`. It fixes the random numbers so your output is repeatable.

■ RECAP

Five Things to Carry Forward

- 1 AdaBoost builds decision tree stumps, depth one, one after another.
- 2 Every stump gets a weight $\alpha = \frac{1}{2} \ln[(1 - TE) / TE]$. Fewer mistakes, bigger weight.
- 3 Wrong rows get a higher weight, correct rows a lower one. Then normalise.
- 4 Bins plus random numbers pull the hard rows into the next stump's table.
- 5 The final answer adds up the α values on each side and takes the bigger total.

Next: Gradient Boosting, the second algorithm in the boosting family.